

UCS503P – Software Engineering Project
Academic Year 2026–2027



RepairFlow: Right-to-Repair Assistant & Reparability Portal

Repair-Shop Discovery, Request Tracking, and a
RAG-Assisted Reparability Portal

Submitted by:

1024030684 Punya Chhabra

1024030780 Arnav Gupta

1024030431 Kaushik Jindal

B.E. Third Year, Computer Science & Engineering

Submitted to:

Dr. Jeelani Asif

Department of Computer Science and Engineering

Mid-Semester Evaluation Report

Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	2
1.1	Background and Context	2
1.1.1	Problem Statement	2
1.2	Project Objectives	2
2	Methodology and Design	3
2.1	System Architecture	3
2.1.1	High-Level Design	3
2.1.2	Technical Stack	3
2.2	Implementation Details	3
2.2.1	Module 1: Core Functionality – Guide Matching & Shop Discovery	3
2.2.2	Module 2: Secondary Features	4
3	Results and Evaluation	5
3.1	Testing Methodology	5
3.2	Performance Metrics	5
3.3	Analysis	5
4	Conclusion	6
4.1	Summary of Work	6
4.2	Key Findings	6
4.3	Future Work and Recommendations	6
4.4	Final Remarks	6

Chapter 1

Introduction

1.1 Background and Context

When a phone, laptop, or household appliance develops a fault, the owner faces two disconnected problems at once: finding a repair shop that can be trusted with the job, and finding out, before committing to it, whether the fault is even worth repairing. Independent repair shops still rely largely on paper logs and phone calls, so once a device is dropped off there is no way for the owner to check status, cost, or timeline without calling directly. Repair manuals and genuine spare parts are also hard for both owners and shops to source, so perfectly repairable devices are often written off simply because nobody could quickly establish that they were.

This gap has consequences beyond individual inconvenience: global e-waste reached 62 million tonnes in 2022, and only 22% of it was properly collected and recycled. Right-to-repair legislation – India’s Right to Repair Portal, the EU’s repair directive, and new U.S. state laws – is pushing manufacturers toward repairable design, but legislation alone does not give an owner a fast way to act on it. RepairFlow is proposed as a web platform that closes this gap: an owner describes a device and a fault, is shown repair guidance and a reparability score, and can then discover, request, and track a verified nearby shop through to completion.

1.1.1 Problem Statement

Device owners and independent repair shops lack a single, transparent platform to manage repairs, access parts and documentation, and make repair the easier choice. Owners have no centralised way to discover verified shops or compare quotes; once a device is dropped off there is no visibility into status, cost, or timeline; and slow, opaque repairs push owners toward replacing a device rather than fixing it, adding to the e-waste problem above. RepairFlow addresses this through a RAG-assisted reparability portal combined with a verified-shop directory and an end-to-end request-tracking workflow.

1.2 Project Objectives

The core objectives of the RepairFlow project comprise:

- **Unified Discovery:** Bring device owners, independent repair shops, and reparability information together on one platform.
- **Reparability Guidance:** Surface repair guides and a reparability score for a described device/fault, using a Retrieval-Augmented Generation (RAG) approach over curated documentation.
- **Verified Shop Directory:** List only shops that have passed a verification process, filterable and sortable by rating, distance, and price.
- **End-to-End Tracking:** Let an owner submit a request and follow it through defined status stages, with a “last updated” indicator so a stale status is never mistaken for current.

Chapter 2

Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

RepairFlow follows a standard three-tier design: the **Client Layer**, the **Application Layer**, and the **Data Layer**. At this mid-semester stage, the Client Layer has been built as a fully interactive, click-through prototype using static in-browser data, while the Application and Data layers are designed but not yet implemented. Figure 2.1 illustrates the architecture.

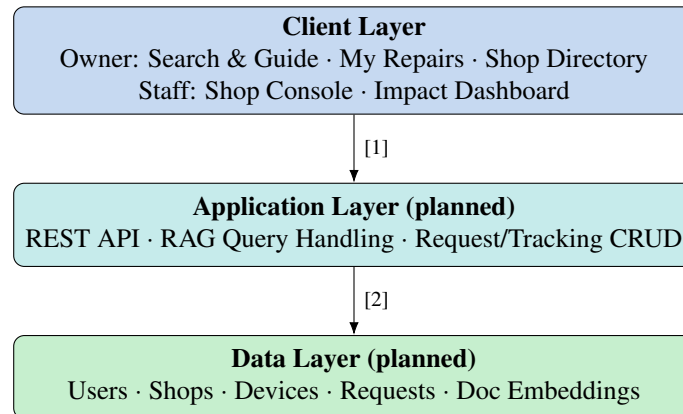


Figure 2.1: RepairFlow Three-Tier Architecture – client layer implemented; application and data layers designed and pending

The core workflow exercised by the prototype is: an owner describes a device or fault → the client matches it against a guide/shop dataset and returns a reparability score, a guide, and a ranked shortlist of verified shops → the owner requests a shop and receives a tracking ID → the shop updates status and cost, which is reflected immediately back in the owner’s tracking view.

2.1.2 Technical Stack

- **Prototype (built):** HTML5, CSS3, and vanilla JavaScript, implemented as a single responsive page with a tab-based router and in-memory state.
- **Planned Backend:** A REST API layer (Spring Boot or an equivalent Node/Express service) handling authentication, request/tracking CRUD, and the shop-verification workflow.
- **Planned Persistence:** A managed relational database for users, shops, devices, and repair requests, alongside a vector store for documentation embeddings used by the RAG pipeline.
- **CI/CD:** A CI runner for build, backend unit tests, and linting on every change, with CD to a staging environment.
- **Version Control:** Git, used to track the prototype and, going forward, the backend implementation.

2.2 Implementation Details

2.2.1 Module 1: Core Functionality – Guide Matching & Shop Discovery

The Search view is the entry point: an owner types a free-text device/fault description, and a keyword-matching function scores the query against a seeded dataset of four repair guides. The best match populates a repair “ticket” card with the device, fault, parts-availability estimate, and a reparability score, and the same query ranks the seeded verified shops to produce a shortlist. Figure 2.2 shows this pipeline.

This keyword matcher is a deliberate placeholder for the RAG pipeline: it validates the intended contract – free-text query in, guide plus score out – before real retrieval and generation are built.

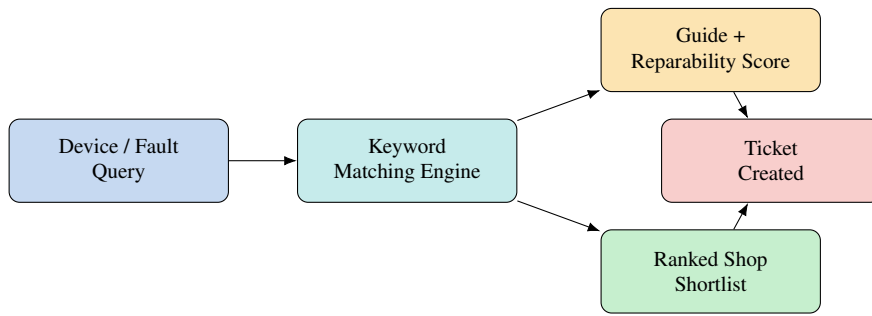


Figure 2.2: Core Guide-Matching and Shop-Discovery Pipeline

2.2.2 Module 2: Secondary Features

Four further views extend the prototype into the fuller workflow the proposal describes. Figure 2.3 shows how a created ticket flows through the remaining views.

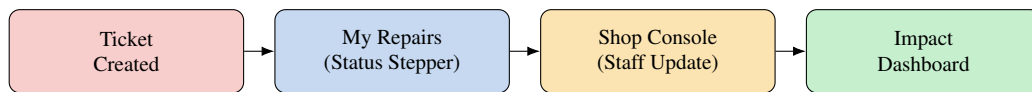


Figure 2.3: Secondary Feature Flow: Tracking, Staff Updates, and Sustainability Reporting

- **My Repairs:** A five-step status stepper (Submitted → Confirmed → In progress → Ready → Closed) per ticket, flagging any open ticket whose “last updated” timestamp exceeds 48 hours as “May be stale.”
- **Shop Directory:** The full seeded shop list, filterable by name/specialty and re-sortable by rating, distance, or price.
- **Shop Console:** A staff-facing table to set a status and cost per ticket; saving writes back into the shared ticket list.
- **Impact Dashboard:** Demo figures for repairs completed and estimated e-waste diverted, shown as placeholders pending real telemetry.

Chapter 3

Results and Evaluation

3.1 Testing Methodology

Because the current build is a client-only prototype, testing at this stage was manual and interaction-driven rather than automated. Each of the five views was exercised directly in the browser: the Search view was tested against all seeded fault categories, the Request → My Repairs handoff was checked for correctness, the Shop Console’s status/cost edits were confirmed to propagate back into My Repairs, and the 48-hour staleness rule was verified against tickets whose timestamps were deliberately set at 3, 52, and 140 hours old.

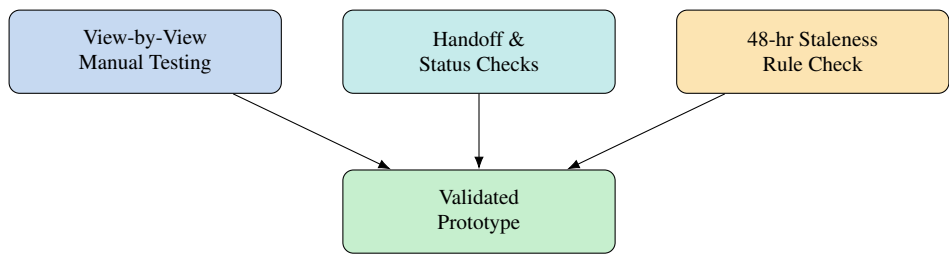


Figure 3.1: Incremental View-by-View Testing Flow

3.2 Performance Metrics

The four evaluation metrics defined for the project require a live backend and real user traffic to measure meaningfully; they cannot yet be captured against a static prototype and are therefore reported as pending. Table 3.1 summarises the current status.

Evaluation Metric	Target Specification	Status
Median time-to-answer for a repair query (guides + matching shops)	≤ 3 s	Pending
Retrieval relevance rate (queries returning ≥ 1 useful guide/shop)	≥ 90%	Pending
Request completion rate (completed vs. abandoned/cancelled)	Tracked, no target yet	Pending
System uptime during business hours (pilot)	≥ 99%	Pending

Table 3.1: RepairFlow Planned Performance Evaluation Metrics

3.3 Analysis

The prototype shows that the intended workflow – describe a fault, receive guidance and a shortlist, request a shop, track it, and roll the outcome into a sustainability figure – is coherent end to end. The keyword-matching placeholder also gives an early, concrete shape to the RAG pipeline’s contract, reducing integration risk once real retrieval and generation are wired in. The remaining work is the Application and Data layers themselves, after which the metrics in Section 3.2 can be measured against real usage.

Chapter 4

Conclusion

4.1 Summary of Work

This project set out to build RepairFlow, a platform that gives device owners transparent access to repair guidance and verified nearby shops while tracking a repair from request to completion. For this mid-semester stage, the team designed the three-tier architecture proposed at kickoff and implemented a fully interactive client-layer prototype covering guide/shop matching, request creation, status tracking with a staleness safeguard, a browsable verified-shop directory, a staff console, and a sustainability dashboard.

4.2 Key Findings

- A small, well-chosen set of fields – reparability score, parts availability, shop rating/distance/price, and a last-updated timestamp – is enough to drive the entire owner-facing workflow.
- Building the client layer against a static, in-memory dataset before the backend exists validated the full interaction flow early and gives a concrete input/output contract for the RAG pipeline to be built against.

4.3 Future Work and Recommendations

For the final project phase, the team will implement:

- Build the backend REST API (authentication, request/tracking CRUD, shop verification) and replace the in-memory dataset with a persistent database.
- Implement the real RAG pipeline: embed curated repair manuals/guides into a vector store and connect an LLM API call in place of the current keyword matcher.
- Implement the shop registration and verification workflow, including lightweight document upload and admin approval.
- Wire the CI pipeline (build, backend unit tests, linting) and CD to a staging environment.

4.4 Final Remarks

RepairFlow addresses a genuine problem: the absence of a single, transparent place to discover a trustworthy repair shop and know whether a device is worth repairing. The prototype shows the intended owner and shop workflows hold together end to end, on a conventional, deployable stack. What remains is building out the Application and Data layers – the real API, database, RAG pipeline, and verification workflow – and then validating accuracy, relevance, and performance at pilot scale.